

PYTHON FUNDAMENTALS

Cloud Infrastructure Monitoring and Cost Analysis

Integrated Coding Assignment

Assignment	01	Total Marks	100
Level	Beginner to Intermediate	Suggested Time	2-3 hours
Module	Python Basics	Submission	Single .py file
Topics	Data types, operators, conditions and loops	Mode	Individual
Collections	Not permitted	External Libraries	Not permitted

Student Details

Name	
Batch / Cohort	
Submission Date	

Build the logic. Validate the input. Explain the operational decision.

1. Assignment Overview

Purpose. This assignment evaluates your ability to combine Python fundamentals into one practical DevOps and cloud operations program. You will build a command-line application that monitors infrastructure usage, evaluates application health, recommends scaling actions, tracks operational events, and estimates cloud cost.

Important: This is a logic-building assignment. Do not use lists, tuples, sets, dictionaries, functions, classes, modules, or external libraries.

Learning Outcomes

- Use int, float, str and bool values correctly.
- Apply arithmetic, comparison, assignment and logical operators.
- Design multi-level decision logic using if, elif and else.
- Use for and while loops for repetition, validation and monitoring.
- Use break and continue for operational control flow.
- Implement counters, accumulators and threshold-based decisions.
- Translate DevOps monitoring and auto-scaling requirements into Python logic.

Permitted Python Features

Category	Allowed
Basic data types	int, float, str, bool
Input and output	input(), print(), type conversion
Operators	+, -, *, /, %, **, //, ==, !=, >, <, >=, <=, and, or, not
Control flow	if, elif, else, for, while, break, continue

Not Permitted

- list, tuple, set or dictionary objects
- user-defined functions or classes
- imports or external libraries such as psutil
- hard-coded output that bypasses the required logic
- copying a complete solution from another learner or AI tool without understanding and explaining the code

2. Business Scenario

Scenario. You are supporting a cloud-hosted application. The application runs on multiple compute instances and must be monitored for CPU, memory, disk, HTTP response status, latency and cost. The program will receive monitoring values for several cycles, classify the system state, perform a simulated auto-scaling decision and produce a final operations report.

3. Problem Statement

Develop a Python program named `cloud_monitor.py` that completes all requirements in Sections 4 to 12.

Assumption: Each valid monitoring cycle represents one hour of cloud usage. Cost must be calculated after the scaling decision for that cycle.

4. Initial Configuration and Validation

At program start, accept the following values from the user:

1. Environment name
2. Number of monitoring cycles
3. Minimum instance count
4. Maximum instance count
5. Current instance count
6. Hourly cost per instance

Environment Validation

The input must be case-insensitive. Only the following environment names are valid:

```
development
testing
staging
production
```

Keep asking for the environment until a valid value is entered. Do not use a collection to perform this validation.

Configuration Validation Rules

Input	Validation Rule	Expected Behaviour
Monitoring cycles	Greater than 0	Repeat input until valid
Minimum instances	At least 1	Repeat input until valid
Maximum instances	Greater than or equal to minimum	Repeat input until valid
Current instances	Between minimum and maximum	Repeat input until valid
Hourly cost	Zero or greater	Repeat input until valid

5. Monitoring Cycle Input

Run the required number of monitoring cycles using a loop. During each cycle, accept:

- CPU usage percentage
- Memory usage percentage
- Disk usage percentage
- HTTP status code
- Application response time in milliseconds

Resource Input Validation

CPU, memory and disk usage must be between 0 and 100 inclusive. If any of these values is invalid, print the following message and skip the remaining logic for that cycle using continue:

```
Invalid resource usage. Monitoring cycle skipped.
```

Counter Rule: A skipped cycle must not affect health counters, application counters, scaling counters, cost or completed-cycle count.

6. Resource Health Classification

Classify the infrastructure resource status using the following priority order:

Status	Condition	Output
Critical	CPU > 80 OR memory > 80 OR disk > 85	Resource status: Critical
Warning	CPU >= 60 OR memory >= 60 OR disk >= 70	Resource status: Warning
Healthy	All remaining valid readings	Resource status: Healthy

Priority: Check Critical before Warning. A reading that satisfies both must be classified as Critical.

7. Application Health Classification

Application Status	Condition	Required Output
Healthy	HTTP 200-299 AND response time <= 500 ms	Application status: Healthy
Slow	HTTP 200-299 AND response time > 500 ms	Application status: Slow
Failed	HTTP status outside 200-299	Application status: Failed

Successful HTTP response counter. Increment for both Healthy and Slow responses because their HTTP status is in the 200-299 range.

8. Simulated Auto-Scaling Decision

Use CPU and memory to recommend an instance-count change. The environment affects the scale-out threshold.

Environment	Scale-Out Threshold
production	CPU > 70 OR memory > 70
development, testing or staging	CPU > 75 OR memory > 75

Scaling Rules

Decision	Condition	Boundary Check	Action
Scale out	Environment-specific scale-out threshold is exceeded	Current instances < maximum	Increase by 1
Maximum reached	Scale-out condition is true	Current instances == maximum	Do not change count
Scale in	CPU < 30 AND memory < 30	Current instances > minimum	Decrease by 1
Minimum reached	Scale-in condition is true	Current instances == minimum	Do not change count
No scaling	No scale-out or scale-in rule is true	Not applicable	Do not change count

Use the following messages as applicable:

```
Scaling action: Scale out
Scaling action: Scale in
Scaling action: Maximum instance limit reached
Scaling action: Minimum instance limit reached
Scaling action: No scaling required
```

9. Counters, Accumulators and Consecutive Failures

Create separate variables to count the following outcomes. Do not use a list or dictionary:

- Healthy resource cycles
- Warning resource cycles
- Critical resource cycles
- Successful HTTP responses
- Slow application responses
- Failed HTTP responses
- Scale-out actions
- Scale-in actions
- Completed valid monitoring cycles

Consecutive Failure Rule

7. When the application status is Failed, increase consecutive_failures by 1.
8. When the application status is Healthy or Slow, reset consecutive_failures to 0.
9. When consecutive_failures becomes 3, display a critical alert and stop the monitoring loop using break.

```
Critical alert: Application failed three times consecutively.
Monitoring stopped.
```

10. Cloud Cost and Budget Analysis

Cost timing. Calculate the cycle cost after applying the scaling decision, using the updated instance count.

```
cycle_cost = current_instances * hourly_cost
total_cost = total_cost + cycle_cost
```

After monitoring ends, ask the user to enter a cloud budget. Validate that the budget is zero or greater, then print:

- Within budget and remaining budget when total cost is less than or equal to the budget.
- Budget exceeded and amount exceeded when total cost is greater than the budget.

11. Overall Infrastructure Status

Final Status	Condition	Priority
Unstable	Monitoring stopped because of 3 consecutive application failures	Highest
Attention Required	At least 1 critical resource cycle OR 1 failed HTTP response OR budget exceeded	Medium
Stable	No critical cycle, no failed HTTP response and cost is within budget	Lowest

12. Final Monitoring Report

At the end of the program, display a clearly formatted report containing at least the following fields:

- Environment
- Requested monitoring cycles and completed valid cycles
- Final instance count
- Healthy, warning and critical resource counts
- Successful, slow and failed application counts
- Scale-out and scale-in action counts
- Estimated total cloud cost
- Cloud budget and remaining/exceeded amount
- Overall infrastructure status

```
=====
CLOUD INFRASTRUCTURE MONITORING REPORT
=====
Environment: production
Completed valid cycles: 4
Final instance count: 3

Healthy resource cycles: 2
Warning resource cycles: 1
Critical resource cycles: 1

Successful HTTP responses: 3
Slow application responses: 1
Failed HTTP responses: 1

Scale-out actions: 2
Scale-in actions: 1

Estimated cloud cost: 120.00
Cloud budget: 150.00
Remaining budget: 30.00

Overall infrastructure status: Attention Required
=====
```

13. Sample Execution

The sample below demonstrates expected interaction and logic. Your exact formatting may differ, but all required decisions and values must be displayed.

```
Enter environment: production
Enter number of monitoring cycles: 4
Enter minimum instances: 2
Enter maximum instances: 5
Enter current instances: 2
Enter hourly cost per instance: 10

-----
Monitoring cycle 1
-----
Enter CPU usage: 45
Enter memory usage: 40
Enter disk usage: 55
Enter HTTP status code: 200
Enter response time in milliseconds: 120
Resource status: Healthy
Application status: Healthy
Scaling action: No scaling required
Current instances: 2
Cycle cost: 20.00
```

```
-----  
Monitoring cycle 2  
-----  
Enter CPU usage: 78  
Enter memory usage: 65  
Enter disk usage: 72  
Enter HTTP status code: 200  
Enter response time in milliseconds: 650  
Resource status: Warning  
Application status: Slow  
Scaling action: Scale out  
Current instances: 3  
Cycle cost: 30.00  
  
-----  
Monitoring cycle 3  
-----  
Enter CPU usage: 90  
Enter memory usage: 85  
Enter disk usage: 88  
Enter HTTP status code: 503  
Enter response time in milliseconds: 900  
Resource status: Critical  
Application status: Failed  
Scaling action: Scale out  
Current instances: 4  
Cycle cost: 40.00  
  
-----  
Monitoring cycle 4  
-----  
Enter CPU usage: 25  
Enter memory usage: 20  
Enter disk usage: 60  
Enter HTTP status code: 200  
Enter response time in milliseconds: 100  
Resource status: Healthy  
Application status: Healthy  
Scaling action: Scale in  
Current instances: 3  
Cycle cost: 30.00  
  
Enter cloud budget: 150
```

14. Minimum Test Cases

Before submission, test your program with all of the following situations:

Test	Example	Expected Focus
Invalid environment	prod	Program asks again
Invalid percentage	CPU = 120	Cycle is skipped with continue
Healthy system	Low CPU, memory and disk	Healthy classification
Critical resource	Memory = 90	Critical overrides warning
Slow application	HTTP 200, latency 700 ms	Slow and successful HTTP counters
Failed application	HTTP 503	Failure and consecutive-failure counters
Maximum instances	Scale-out condition at maximum	No count increase
Minimum instances	Scale-in condition at minimum	No count decrease
Three failures	Three failed HTTP cycles in sequence	Loop stops with break
Budget exceeded	Total cost > budget	Exceeded amount and Attention Required

15. Evaluation Rubric

Assessment Area	Criteria	Marks
Input and configuration validation	Correct while-loop validation for environment and numeric configuration	15
Resource health logic	Correct thresholds, priority and counters	10
Application health logic	Correct HTTP/latency classification and counters	10
Auto-scaling logic	Correct environment threshold, scale-in/out and min/max boundaries	15
Loop control	Correct use of for/while, continue and break	15
Counters and failure tracking	Accurate counters and consecutive-failure reset/stop logic	15
Cost and budget analysis	Correct per-cycle cost, total, remaining/exceeded calculations	10
Final report and code quality	Readable variable names, indentation, comments and complete report	10

Total: 100 marks

16. Submission Guidelines

10. Submit one Python file only.
11. Use the filename format: FirstName_LastName_CloudMonitor.py
12. The program must run from the command line without modification.
13. Do not submit code containing lists, tuples, sets, dictionaries, functions, classes or imported libraries.
14. Include short comments before major sections of logic.
15. Use meaningful variable names and consistent indentation.
16. Test the program using the minimum test cases provided in Section 14.

Submission Checklist

- ☐ The program accepts and validates all configuration values.
- ☐ All monitoring values are processed using loops and conditions.
- ☐ Invalid resource readings are skipped correctly.
- ☐ Health, application and scaling decisions follow the stated thresholds.
- ☐ Three consecutive failures stop monitoring.
- ☐ Cost is calculated after the scaling decision.
- ☐ The final report contains every required field.
- ☐ No prohibited data structure or Python feature is used.

Academic Expectation: You may consult Python documentation and class notes. You must be able to explain every condition, loop, counter and calculation used in your submission.

END OF ASSIGNMENT